Escenarios de Calidad - Planazo (Sprint 2)

Escenarios de Calidad - Planazo (Sprint 2)

Introducción

Este documento consolida los escenarios de calidad exigidos en el Proyecto Sprint 2 para Planazo, la plataforma de microservicios para salir de noche en Bogotá (Zona G y Zona T): mapa en vivo de cupos, reservas, promociones de cupo limitado, el minijuego multijugador Snake y un agente con IA que arma rutas. La arquitectura se compone de api-gateway, booking, promo, realtime y Minijuego, con PostgreSQL, Redis como bus de eventos y caché, y el frontend en Vercel.

El enunciado exige **7 escenarios de calidad** distribuidos en 5 atributos: Seguridad (3), Disponibilidad (1), Mantenibilidad (1), Rendimiento/Latencia (1) y Observabilidad (1). Cada escenario sigue la plantilla de 6 partes de Bass, Clements y Kazman (Fuente del estímulo, Estímulo, Artefacto, Ambiente, Respuesta, Medida de respuesta) y se complementa con la táctica arquitectónica aplicada, los componentes que la implementan y la forma de demostrarlo.

Los 3 escenarios de Seguridad y el escenario de Rendimiento/Latencia requieren aprobación del profesor; el de Rendimiento, a más tardar el 13 de noviembre. Este documento se entrega para esa revisión.

Arquitecturas de Software 2026-2 · Escuela Colombiana de Ingeniería Julio Garavito · Equipo Spidiverse: Juan Diego Melo Suárez, Diego Fabián Andrade Durán, Diego Alejandro Rozo Gaviria y Juan Camilo Melo Cupitra.

Resumen de escenarios

#	Atributo	Escenario	Táctica principal	Aprobación
ESC-01	Seguridad	Autenticación de cuentas de negocio	Autenticar actores y resistir ataques (bloqueo de cuenta)	Requiere aprobación
ESC-02	Seguridad	Autorización de peticiones al Minijuego	Autorizar actores (control de acceso stateless)	Revisado por el profesor; ajustado según sus comentarios
ESC-03	Seguridad	Detección de ataques y trazabilidad	Detectar ataques y recuperarse de ataques (auditoría)	Requiere aprobación
ESC-04	Disponibilidad	Escalamiento horizontal y tolerancia a fallos	Redundancia activa, balanceo de carga, detección de fallos (health checks) y reintento	No requiere aprobación previa
ESC-05	Mantenibilidad	Inspección continua con análisis estático	Inspección continua (análisis estático en el pipeline de CI)	No requiere aprobación previa
ESC-06	Rendimiento / latencia	Entrega de eventos en tiempo real bajo concurrencia	Procesamiento asíncrono (bus de eventos), caché (Redis), escalado horizontal y pool de conexiones	Requiere aprobación (fecha límite: 13 de noviembre)

#	Atributo	Escenario	Táctica principal	Aprobación
ESC-07	Observabilidad	Detección y diagnóstico de degradaciones	Monitorear: logs estructurados, métricas y tableros de negocio y técnico	No requiere aprobación previa

Ajustes según la retroalimentación del profesor

Sobre el escenario «Autenticación y autorización de peticiones» enviado por el equipo, el profesor indicó que autenticación y autorización son aspectos distintos de la seguridad y deben ir en escenarios separados, y que el escenario tenía demasiadas fuentes de estímulo, muy diferentes entre sí. En consecuencia:

• **ESC-01 (autenticación)** es un escenario nuevo: verificar la identidad de quien inicia sesión como negocio.

• **ESC-02 (autorización)** conserva el artefacto, la respuesta y la medida del escenario revisado, pero ahora tiene una sola fuente: un jugador ya autenticado que intenta actuar a nombre de otro.

• Todos los escenarios de este documento tienen **una única fuente y un único estímulo**.

ESC-01 · Seguridad: Autenticación de cuentas de negocio

Táctica: Autenticar actores y resistir ataques (bloqueo de cuenta) · **Aprobación:** Requiere aprobación

Fuente del estímulo	Un atacante externo que conoce el nombre de una cuenta de negocio registrada en Planazo.
Estímulo	Envía 10 intentos de inicio de sesión con contraseñas incorrectas en 1 minuto a POST /api/auth/negocio para tomar el control del panel del negocio.
Artefacto	Servicio de autenticación del api-gateway (emisor del JWT) y almacén de cuentas en el microservicio booking (hash bcrypt + contador de intentos en Redis).
Ambiente	Operación normal, con otros negocios y clientes iniciando sesión al mismo tiempo.
Respuesta	El sistema valida la contraseña contra el hash bcrypt; ante cada fallo responde con un error genérico que no revela si la cuenta existe ni qué dato falló; al 5.º intento fallido bloquea la cuenta 15 minutos y registra el evento en la bitácora de auditoría. Con credenciales válidas emite un JWT firmado (HS256) con el rol negocio.
Medida de respuesta	100 % de los intentos con credenciales inválidas rechazados con 401 en menos de 300 ms; la cuenta queda bloqueada desde el 5.º intento fallido; 0 accesos indebidos; las contraseñas se guardan solo como hash bcrypt (cost ≥ 10).

Cómo lo atiende la arquitectura: Contraseña con bcrypt para cuentas de negocio, contador de intentos fallidos en Redis (compartido por todas las instancias del gateway), mensajes de error genéricos y JWT firmado por el gateway.

Cómo se demuestra: Script automatizado de fuerza bruta contra producción que verifica el bloqueo en el 5.º intento y mide los tiempos de respuesta.

ESC-02 · Seguridad: Autorización de peticiones al Minijuego

Táctica: Autorizar actores (control de acceso stateless) · **Aprobación:** Revisado por el profesor; ajustado según sus comentarios

Fuente del estímulo	Un jugador autenticado, con un JWT válido y vigente, que intenta actuar a nombre de otro jugador.
Estímulo	Invoca un endpoint protegido del Minijuego con un jugadorId que no es el suyo (por ejemplo, unirse a una sala con POST /api/v1/salas/{salaId}/jugadores o consultar el histórico de partidas de otro jugador), o intenta enviar comandos por STOMP a una partida en la que no está inscrito.
Artefacto	Capa de autorización stateless del microservicio Minijuego: filtro de Spring Security OAuth2 Resource Server (SecurityConfig) para REST e interceptor StompAuthChannelInterceptor para WebSocket/STOMP.
Ambiente	Operación normal, con tráfico concurrente (varios jugadores reales conectados a la vez).
Respuesta	El sistema compara el jugadorId de la petición con el sujeto (sub) del JWT y rechaza la petición antes de que toque cualquier lógica de negocio: no se crea ningún efecto secundario (el jugador no queda unido a la sala, el comando no se encola) y no se revela si el recurso solicitado existe.
Medida de respuesta	100 % de las peticiones a nombre de otro jugador rechazadas con 403 Forbidden (REST) o con un frame ERROR (STOMP) en menos de 200 ms; 0 invocaciones a la capa de dominio, verificado con pruebas automatizadas.

Cómo lo atiende la arquitectura: Validación del JWT en el Resource Server, regla jugadorId == jwt.sub en los endpoints y en el canal STOMP, respuestas uniformes que no revelan la existencia del recurso.

Cómo se demuestra: Pruebas de integración (MockMvc y cliente STOMP) y script contra producción con un token válido de otro jugador; se mide el % rechazado y el tiempo de respuesta.

ESC-03 · Seguridad: Detección de ataques y trazabilidad

Táctica: Detectar ataques y recuperarse de ataques (auditoría) · **Aprobación:** Requiere aprobación

Fuente del estímulo	Un script automatizado que ataca Planazo desde una misma dirección IP.
Estímulo	Genera más de 50 respuestas 401/429 en 1 minuto (credenciales o tokens rechazados y peticiones por encima del rate limit).
Artefacto	Detector de patrones del api-gateway, bitácora de auditoría (tabla audit_log, append-only) y tablero técnico de observabilidad.
Ambiente	Operación normal, con tráfico real de usuarios mezclado con el tráfico malicioso.
Respuesta	El sistema detecta el patrón, bloquea la IP durante 10 minutos sin afectar a los usuarios legítimos, emite una alerta en el tablero técnico y deja rastro. Toda operación sensible (inicio de sesión, crear o editar una promoción, cambiar la capacidad de un lugar, reclamar un cupón) queda en la bitácora con quién, qué, cuándo, IP y correlation-id.
Medida de respuesta	Detección y bloqueo de la IP en 60 s o menos; alerta visible en el tablero técnico en 60 s o menos; 100 % de las operaciones sensibles auditadas; los registros de la bitácora no se pueden modificar ni borrar desde la aplicación.

Cómo lo atiende la arquitectura: Contadores por IP en Redis con ventana de 1 minuto, lista de IP bloqueadas en Redis, interceptor de auditoría y tabla audit_log sin permisos de UPDATE/DELETE para el usuario de la aplicación.

Cómo se demuestra: Script de ataque simulado contra producción; se verifica el bloqueo, la alerta y los registros de auditoría generados.

ESC-04 · Disponibilidad: Escalamiento horizontal y tolerancia a fallos

Táctica: Redundancia activa, balanceo de carga, detección de fallos (health checks) y reintento · **Aprobación:** No requiere aprobación previa

Fuente del estímulo	Un fallo interno: una instancia del api-gateway deja de funcionar (caída del proceso o del contenedor).
Estímulo	La instancia deja de responder mientras atiende tráfico.
Artefacto	Balanceador de carga (Nginx) y las instancias redundantes del backend (api-gateway, booking, promo y realtime) desplegadas en Railway.
Ambiente	Carga pico: 300 usuarios concurrentes (viernes 10 p. m.), con al menos 2 instancias activas de cada servicio detrás del balanceador.
Respuesta	El balanceador detecta la falla con sus health checks, saca la instancia de rotación y reparte el tráfico entre las instancias sanas; Railway reinicia la instancia caída. Los clientes WebSocket afectados reconectan solos y, con RESUME, reciben los eventos que se perdieron.
Medida de respuesta	Tasa de error menor al 1 % durante la prueba; la instancia caída sale de rotación en 10 s o menos; reconexión WebSocket en 5 s o menos; 0 eventos perdidos o duplicados; disponibilidad del 99 % o más durante la ventana de prueba.

Cómo lo atiende la arquitectura: Nginx como balanceador explícito (restricción tecnológica), servicios sin estado (estado compartido en PostgreSQL y Redis), fan-out de eventos por Redis entre instancias de realtime, seq global para RESUME, restart policy en Railway. Frontend en Vercel; backend, PostgreSQL y Redis en nodos distintos.

Cómo se demuestra: Prueba de caos con k6 (100–300 usuarios virtuales) en la que se apaga una instancia en plena carga; se mide la tasa de error, el tiempo fuera de rotación y la reconexión.

ESC-05 · Mantenibilidad: Inspección continua con análisis estático

Táctica: Inspección continua (análisis estático en el pipeline de CI) · **Aprobación:** No requiere aprobación previa

Fuente del estímulo	Un desarrollador del equipo.
Estímulo	Abre un pull request hacia la rama main de cualquiera de los repositorios de Planazo.
Artefacto	Pipeline de integración continua (GitHub Actions, ci.yml de cada repositorio) con el análisis de SonarCloud integrado.
Ambiente	Flujo normal de desarrollo, antes de fusionar el cambio.
Respuesta	El pipeline compila, ejecuta las pruebas con reporte de cobertura (lcov en TypeScript, JaCoCo en Java) y corre el análisis estático de SonarCloud. El resultado del Quality Gate se publica en el pull request y bloquea la fusión si falla.
Medida de respuesta	100 % de los pull requests analizados antes de fusionarse, con resultado en 10 minutos o menos; calificación A en confiabilidad, seguridad y mantenibilidad; cobertura de al menos 80 % en el backend y 40 % en el frontend; duplicación de código de 3 % o menos.

Cómo lo atiende la arquitectura: Paso de SonarCloud en el ci.yml de los 7 repositorios, Quality Gate como check obligatorio en los pull requests, reportes de cobertura por repositorio.

Cómo se demuestra: Pull request de prueba que introduce un code smell crítico y baja la cobertura; se verifica que el Quality Gate lo bloquea.

ESC-06 · Rendimiento / latencia: Entrega de eventos en tiempo real bajo concurrencia

Táctica: Procesamiento asíncrono (bus de eventos), caché (Redis), escalado horizontal y pool de conexiones · **Aprobación:** Requiere aprobación (fecha límite: 13 de noviembre)

Fuente del estímulo	500 clientes conectados por WebSocket al mapa en vivo, suscritos a la zona zone:zona-t.
Estímulo	Un negocio de la Zona T lanza una promoción relámpago, lo que genera eventos PROMO.PUSH y PLACE.UPDATE que deben llegar a todos los suscritos.
Artefacto	Flujo de tiempo real: microservicio promo, bus de eventos en Redis y microservicio realtime (WebSocket).
Ambiente	Carga pico: 500 conexiones WebSocket activas en producción.
Respuesta	El evento se publica en el bus y realtime lo entrega a todos los clientes suscritos, sin bloquear el procesamiento de las demás peticiones; las lecturas frecuentes (catálogo de lugares) se sirven desde Redis sin consultar PostgreSQL.
Medida de respuesta	Latencia publicación → entrega de 300 ms o menos en el percentil 95 (y 500 ms o menos en el p99); 0 eventos perdidos; tasa de error menor al 1 %.

Cómo lo atiende la arquitectura: Bus de eventos en Redis (publicación asíncrona), realtime escalado a varias instancias con fan-out, timestamp de emisión en cada evento, caché del catálogo en Redis con invalidación por PLACE.UPDATE.

Cómo se demuestra: Prueba de carga con k6 (500 conexiones WebSocket) que mide la latencia de cada evento; reporte con p50, p95 y p99, comparando 1 y 2 instancias de realtime.

ESC-07 · Observabilidad: Detección y diagnóstico de degradaciones

Táctica: Monitorear: logs estructurados, métricas y tableros de negocio y técnico · **Aprobación:** No requiere aprobación previa

Fuente del estímulo	El equipo de operación de Planazo.
Estímulo	Una instancia de booking empieza a fallar y la latencia de las reservas sube en producción.
Artefacto	Logs estructurados con correlation-id, endpoint /metrics de cada servicio (prom-client y Micrometer), recolector de métricas y los dos tableros en Grafana (negocio y técnico).
Ambiente	Producción, operación normal.
Respuesta	El tablero técnico muestra la degradación y dispara una alerta; el equipo identifica el servicio y la instancia afectados y, filtrando los logs por correlation-id, encuentra la causa. El tablero de negocio sigue mostrando usuarios activos, reservas, cupones reclamados y partidas jugadas.
Medida de respuesta	Alerta en 1 minuto o menos; los tableros reflejan los datos con menos de 15 s de retraso; diagnóstico del servicio, la instancia y la causa en 5 minutos o menos; 100 % de los servicios con logs JSON y métricas expuestas.

Cómo lo atiende la arquitectura: Logs JSON (pino y logback) con service, instance y correlationId propagado por HTTP y por el bus; métricas técnicas (latencia, errores, conexiones WebSocket, CPU y memoria por instancia) y de negocio; alertas por tasa de error, instancia caída y latencia.

Cómo se demuestra: Durante la prueba de caos de ESC-04 se verifica la alerta, el tiempo de diagnóstico y los datos de ambos tableros.